Week 2 Lab: Motor Step Response and First-Order Model

ENGR 3303 – Controls Engineering
Fall 2026

Main goal. Use the same Arduino, L298N motor driver, and encoder gearmotor from Week 1 to record motor RPM directly into Python, save the experiment as a CSV file, estimate the first-order model parameters K and τ, and compare the measured motor response with the model.

Learning Outcomes

By the end of this lab, you should be able to:

- collect time-stamped RPM data from Arduino through a serial connection;
- save experimental data automatically as a CSV file;
- identify the steady-state speed of the motor;
- estimate the first-order gain K and time constant τ;
- create $G(s) = K/(\tau s + 1)$;
- compare a measured response with a Python model; and
- explain why a simple first-order model is useful even when it is not perfect.

Hardware and Software

Each team will use:

- Arduino Uno or compatible board;
- L298N motor-driver board;
- 12 V encoder gearmotor and wheel;
- instructor-provided external motor power supply;
- USB cable;
- Arduino IDE;
- Python with NumPy, SciPy, Matplotlib, pandas, and pyserial.

Safety and wiring rules

- Turn motor power OFF before changing wiring.
- Keep fingers, wires, and clothing away from the spinning wheel.
- Do not intentionally stall the motor.
- Arduino GND and L298N GND must be connected together.
- Do not connect the encoder to 12 V.
- Close Arduino Serial Monitor before Python attempts to open the serial port.

Motor and Encoder Wiring

Use the same wiring established in Week 1.

Wire / signal	Function	Connection
Red motor wire	Motor lead	L298N OUT1
White motor wire	Motor lead	L298N OUT2
Blue encoder wire	Encoder power positive	Arduino 5 V
Black encoder wire	Encoder ground	Arduino GND
Yellow encoder wire	Encoder A	Arduino D2
Green encoder wire	Encoder B	Arduino D3
Arduino D9	PWM command	L298N ENA; remove ENA jumper
Arduino D7	Direction command	L298N IN1
Arduino D8	Direction command	L298N IN2
Arduino GND	Common ground	L298N GND

Encoder calibration. The Arduino program counts rising edges on Encoder A only. The course motor has been experimentally calibrated at $\boxed{494.0}$ counts per output-shaft revolution. Use this value throughout the lab unless the instructor changes the hardware or counting method.

Python Package Setup

Install the free Python packages before lab:

```
pip install pyserial pandas numpy matplotlib scipy
```

Lab 2A – Record a Real Motor Step Response

Suggested time: 75 minutes

Part A1 – Understand the experiment

A step response shows how the motor speed changes after the command suddenly changes.



For this lab, Arduino will wait for Python to send the command `START`. After receiving `START`, Arduino will:

1. begin the experiment clock at $t = 0$;
2. keep the motor OFF for 2.0 s;
3. apply the selected PWM command at $t = 2.0$ s;
4. calculate RPM every 0.10 s;
5. send `time,rpm` data to Python;
6. stop the motor automatically after 10 s total; and
7. send `END` to Python.

Part A2 – Select the PWM step

Use a PWM level that reliably starts the motor while leaving room below full command. Unless your instructor assigns another value, begin with

$$\boxed{U_0 = 180 \text{ PWM units.}}$$

Record your actual value:

$$\boxed{U_0 = _____ \text{ PWM units.}}$$

Part A3 – Prepare the Arduino program

Open and upload

`week2_motor_step_response.ino`

Before uploading, check these two lines:

```
const float COUNTS_PER_OUTPUT_REV = 494.0;
const int STEP_PWM = 180;
```

Keep the calibrated CPR at 494.0 unless the instructor changes the motor or encoder-counting method. Change the PWM step only if instructed.

> **Checkpoint 1.** After uploading, close Arduino Serial Monitor. Python and Serial Monitor cannot use the same COM port at the same time.

Part A4 – Find the Arduino COM port

In Arduino IDE, identify the port from

Tools → Port.

Write it here:

Arduino port: _____

Examples are `COM3`, `COM5`, and `COM7`.

Part A5 – Open the Python data logger

Open

`week2_record_motor_data.py`

At the top of the file, set your Arduino port:

```
PORT = "COM3"        # change to your Arduino port
BAUD = 115200
```

The Python program will:

1. open the Arduino serial port;
2. send `START`;
3. receive each `time,rpm` line;
4. store the measurements in arrays;
5. stop when Arduino sends `END`;
6. save the data as a CSV file; and
7. plot RPM versus time automatically.

Part A6 – Run Trial 1

Follow these steps in order:

1. Confirm the wheel is free to rotate.
2. Turn ON the external motor power supply.
3. Confirm Arduino Serial Monitor is closed.
4. In Python, run `week2_record_motor_data.py`.
5. Enter trial number `1` when prompted.
6. Do not touch the wheel during the experiment.
7. Watch the motor remain off for 2 s and then accelerate.
8. Wait until Python reports that the data file has been saved.
9. Turn motor power OFF between trials.
   The file should be named

   `motor_step_trial1.csv`.

   A typical file will look like:

Time_s	RPM
0.10	0.0
0.20	0.0
⋮	⋮
2.00	0.0
2.10	increasing
2.20	increasing
⋮	⋮
9.90	near steady state

Part A7 – Inspect the measured plot

Your plot should show:

- approximately zero RPM before the step;
- a smooth increase after $t = 2$ s;
- an approximately constant final RPM.
  If the plot is clearly incorrect, use the troubleshooting section before collecting more trials.

Part A8 – Repeat the experiment

Collect two additional trials using the same PWM command:

- `motor_step_trial2.csv`
- `motor_step_trial3.csv`

  Do not change PWM between these three trials.

Trial	PWM	Approx. final RPM	Notes
1			
2			
3			

Checkpoint 2. Show the instructor three saved CSV files and three reasonable step-response plots before beginning model identification.

Lab 2B – Identify a First-Order Motor Model

Suggested time: 75 minutes

Part B1 – Choose a representative trial

Use the trial with the cleanest and most typical response. Do not select a trial simply because it gives the answer you expect.

Record the selected trial:

$$\boxed{\text{Selected trial: } _____}$$

Part B2 – Shift time so the PWM step occurs at zero

Arduino applies the step at

$$t_{step} = 2.0 \text{ s}.$$

For the model, define

$$\boxed{t_m = t - 2.0.}$$

Only data at and after the PWM step are used for the model response.

Part B3 – Estimate the initial and steady-state speeds

The initial speed should be close to zero. Python will estimate the baseline speed y_0 using data before the step.

The steady-state speed y_{ss} is estimated by averaging several samples near the end of the experiment.

Record:

$$y_0 = _____ \text{ RPM}, \qquad y_{ss} = _____ \text{ RPM}.$$

Part B4 – Calculate the gain K

For a step from input 0 to U_0,

$$\Delta u = U_0.$$

The output change is

$$\Delta y = y_{ss} - y_0.$$

Therefore,

$$\boxed{K = \frac{\Delta y}{\Delta u} = \frac{y_{ss} - y_0}{U_0}.}$$

Your result:

$$K = _____ \frac{\text{RPM}}{\text{PWM unit}}.$$

Part B5 – Estimate the time constant τ

For a first-order response, one time constant occurs when 63.2% of the total output change has occurred.

Calculate the target speed:

$$\boxed{y_\tau = y_0 + 0.632(y_{ss} - y_0).}$$

Record:

$$y_\tau = _____ \text{ RPM.}$$

From the measured response, determine how long after the PWM step the motor first reaches approximately y_τ:

$$\boxed{\tau = _____ \text{ s.}}$$

Part B6 – Write the motor transfer function

Use

$$\boxed{G(s) = \frac{K}{\tau s + 1}.}$$

Your model:

$$\boxed{G(s) = \frac{_____}{_____ s + 1}}$$

Part B7 – Use the Python analysis program

Open

`week2_analyze_motor_model.py`.

Set:

```
TRIAL_NUMBER = 1    # change to your selected trial
PWM_STEP = 180      # change to the PWM used in your test
```

Run the script. Python will:

1. load the CSV file;
2. estimate y_0 and y_{ss};
3. calculate K;
4. calculate the 63.2% target;
5. estimate τ;
6. create $G(s) = K/(\tau s + 1)$;
7. simulate the model; and
8. plot measured and simulated responses together.

Part B8 – Check the Python result by hand

Do not use Python as a black box. Compare the calculated values with your plot and hand calculations.

Quantity	Hand estimate	Python	Agreement?
Steady-state RPM			
Gain K			
Time constant τ			
Settling time 4τ			

Part B9 – Compare measured and model responses

The Python figure should contain:

- measured RPM data as points;
- the first-order model as a smooth line;
- time after the step on the horizontal axis;
- RPM on the vertical axis.

Evaluate the model:

Feature	Measured	Model	Comment
Final speed (RPM)			
Approx. rise behavior			
Approx. settling time (s)			
Overall match			

> **Remember:** A model does not need to match every encoder fluctuation. The goal is to capture the main relationship between PWM input, final speed, and response speed.

Questions to Submit

Answer briefly but clearly.

1. What PWM step did your team use?

2. What was the measured initial speed y_0?
3. What steady-state speed y_{ss} did you measure?
4. Calculate the gain K.
5. Calculate the 63.2% target speed y_τ.
6. What time constant τ did you estimate?
7. Write your motor transfer function $G(s)$.
8. Estimate the settling time using $t_s \approx 4\tau$.
9. Did the model reach approximately the correct final speed?
10. Did the model rise faster, slower, or about the same as the real motor?
11. If the model final speed is wrong, which parameter should you check first: K or τ? Explain.
12. If the model responds too quickly, should τ be increased or decreased?
13. Give two reasons the real motor may not perfectly match a first-order model.
14. Why is direct Python logging better for this experiment than manually copying RPM values from Serial Monitor?

Troubleshooting

Problem	Check
Python says COM port is unavailable	Close Arduino Serial Monitor and any other program using the port.
Python receives no data	Confirm the correct COM port and 115200 baud. Re-upload Arduino code if needed.
Motor never starts	Confirm 12 V motor power, ENA jumper removal, ENA-D9 connection, and PWM value.
RPM remains zero	Check encoder power, ground, yellow Encoder-A wire to D2, and CPR constant.
RPM is much too high or low	Verify `COUNTS_PER_OUTPUT_REV`.
RPM plot is noisy	Make sure wiring is secure. Small fluctuations are normal; the analysis script uses a short moving average only for estimating τ.
Motor runs backward	Motor direction does not affect this speed-magnitude lab; if needed, power OFF and swap motor leads or reverse IN1/IN2 logic.

What to Submit

One submission per team:

- three CSV data files;
- three measured step-response plots;
- identification of the representative trial;
- hand calculations for y_{ss}, K, y_τ, and τ;
- estimated transfer function;
- measured-vs-model Python plot;
- completed comparison tables;
- answers to Questions 1–14;
- Arduino code;
- Python data-recording and model-analysis code.

End-of-Lab Shutdown

1. Confirm the motor has stopped.

2. Turn OFF the external motor power supply.
3. Save all CSV and Python files.
4. Disconnect USB only after the motor supply is off.
5. Leave the hardware organized for the next lab.